

Relatório 19 - Prática: Pipelines de Dados II - Airflow (III)

Igor Carvalho Marchi

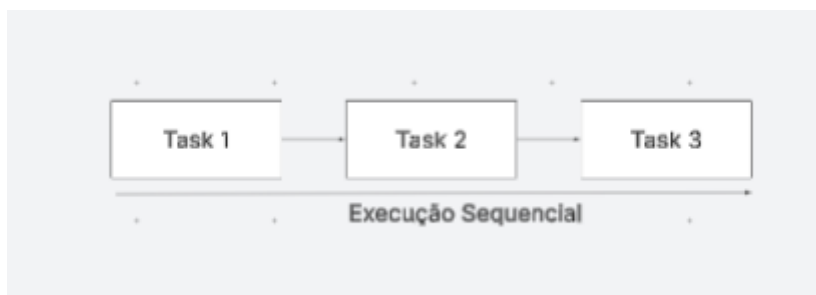
1. Introdução

Neste Card foram aprofundados os estudos sobre Apache Airflow, abordando execução, distribuição e monitoramento de pipelines. Também foram apresentados recursos avançados, como Branching, XCom, TriggerDagRunOperator e ExternalTaskSensor, que foram aplicados em uma atividade prática.

2. Desenvolvimento

2.1 SQLite e Sequential Executor

O SQLite é um banco de dados simples e leve que pode ser utilizado pelo Airflow principalmente em ambientes de teste. Ele possui limitações quando existem várias operações acontecendo ao mesmo tempo, por isso não é a melhor opção para ambientes maiores. O Sequential Executor executa uma tarefa de cada vez. Dessa forma, uma task precisa terminar para que a próxima possa iniciar. É uma configuração simples e adequada principalmente para testes locais.

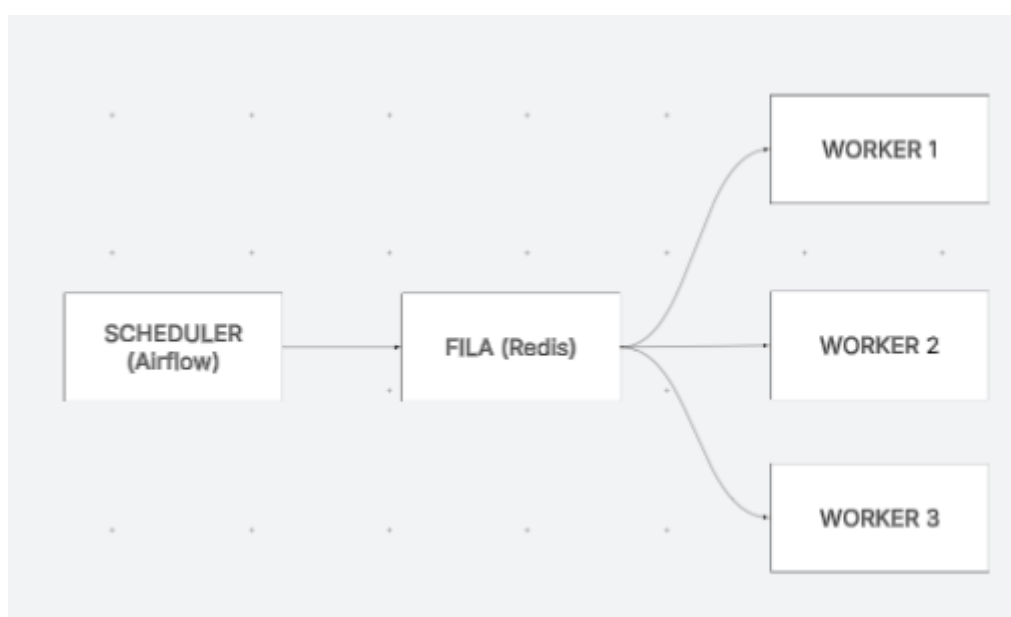


2.2 Local Executor

O Local Executor permite que várias tarefas sejam executadas em paralelo na mesma máquina. Diferente do Sequential Executor, ele consegue aproveitar melhor os recursos disponíveis quando existem várias tasks para executar.

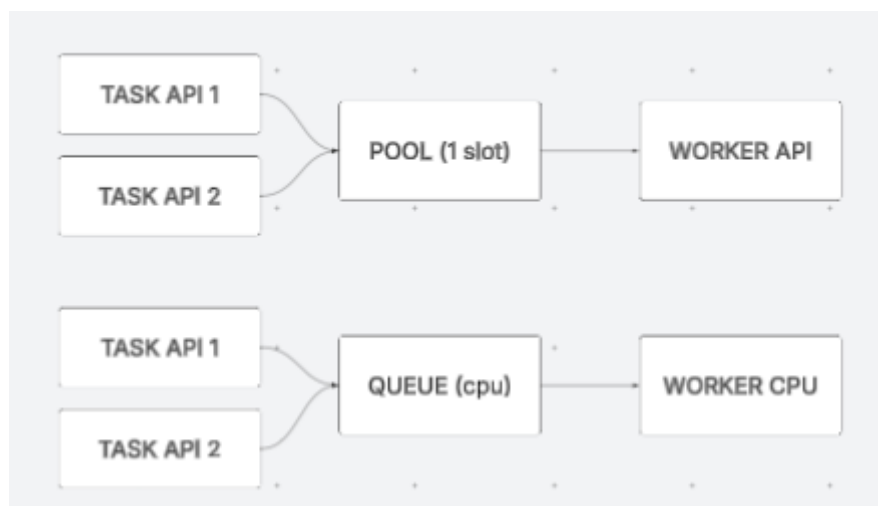
2.3 Celery Executor

O Celery Executor permite distribuir as tarefas entre diferentes workers. Em vez de depender somente de uma máquina para executar todas as tasks, o trabalho pode ser dividido. Nesse funcionamento existe também uma fila que recebe as tarefas que precisam ser executadas. Os workers consultam essa fila e realizam o processamento. O material também apresentou o Flower como ferramenta para acompanhar os workers do Celery.



2.4 Pools e Queues

Pools podem ser utilizados para limitar a quantidade de tarefas que acessam determinado recurso simultaneamente. Isso é útil, por exemplo, quando uma API possui limite de requisições. As Queues permitem direcionar determinadas tarefas para workers específicos. Assim, uma tarefa que exige mais processamento pode ser enviada para um worker preparado para CPU, enquanto outra pode utilizar outro worker.



2.5 Kubernetes Executor

O Kubernetes é uma plataforma utilizada para gerenciar containers. Quando utilizado com o Airflow, permite criar recursos de acordo com a necessidade das tarefas. Uma diferença em relação ao Celery é que os workers do Celery podem permanecer disponíveis aguardando novas tarefas, enquanto no Kubernetes os recursos podem ser criados conforme a necessidade e removidos posteriormente.

2.6 SubDAGs

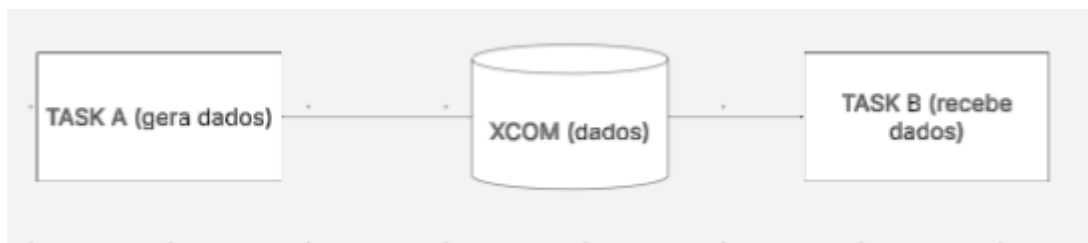
As SubDAGs permitem organizar um conjunto de tarefas dentro de outra DAG. Isso ajuda na organização de pipelines maiores. Porém, é necessário tomar cuidado com os recursos disponíveis. Se várias SubDAGs ocuparem todos os slots disponíveis enquanto suas tarefas internas também precisam desses slots, pode ocorrer um deadlock, impedindo que o fluxo continue. Esse problema foi demonstrado no conteúdo do Card.

2.7 Branching

Branching permite criar caminhos diferentes dentro de uma DAG, com o BranchPythonOperator, uma função pode analisar determinada condição e retornar qual tarefa deverá ser executada. Assim, nem todas as execuções da DAG precisam seguir exatamente o mesmo caminho. Esse conceito também foi utilizado na minha prática para verificar os sensores da fazenda.

2.8 XCom

O XCom permite compartilhar pequenas informações entre tarefas, uma task pode gerar um resultado e outra pode recuperar esse valor posteriormente. Ele é útil para compartilhar informações como IDs, resultados ou caminhos de arquivos, mas não é indicado para armazenar grandes quantidades de dados.



2.9 TriggerDagRunOperator

O TriggerDagRunOperator permite que uma DAG inicie outra DAG, isso é útil quando queremos dividir um processo grande em diferentes fluxos. Uma DAG realiza determinada etapa e, quando necessário, dispara outra responsável pela continuação do processamento. Esse recurso foi importante na prática desenvolvida, pois a DAG de monitoramento pode acionar a DAG de emergência.

2.10 ExternalTaskSensor

O ExternalTaskSensor é utilizado quando uma DAG precisa aguardar uma tarefa pertencente a outra DAG. Enquanto o Trigger inicia outro fluxo, o sensor pode acompanhar uma tarefa externa e continuar somente quando a condição esperada for atendida. No conteúdo estudado, esses dois recursos são apresentados justamente como formas de comunicação e dependência entre DAGs.

2.11 Monitoramento

O Airflow também possui recursos para acompanhar o funcionamento dos pipelines. Os logs permitem verificar o que aconteceu durante a execução de cada tarefa e ajudam principalmente na identificação de erros. Também podem ser utilizadas métricas para acompanhar informações como quantidade de tarefas em execução, tarefas aguardando e tempo de processamento. O conteúdo apresenta ferramentas como InfluxDB e Grafana para armazenamento e visualização dessas métricas.

PRÁTICA

Na atividade prática foi desenvolvido um sistema de monitoramento de uma fazenda utilizando Apache Airflow, dividido em três DAGs responsáveis pelo monitoramento, atendimento de emergências e registro das ocorrências. O sistema simula dados de temperatura, água e ração, analisa os valores e, caso encontre alguma situação fora do normal, aciona automaticamente o fluxo de emergência, realiza as verificações necessárias e registra a ocorrência. A prática utilizou recursos como PythonOperator, BranchPythonOperator, XCom, TaskGroup, EmptyOperator e TriggerDagRunOperator.

3. Conclusão

O Card 19 aprofundou os conhecimentos sobre Apache Airflow, apresentando recursos para execução, distribuição e controle de tarefas. Na prática, foi desenvolvido um sistema de monitoramento de uma fazenda, utilizando diferentes DAGs para analisar dados, identificar emergências e registrar ocorrências automaticamente.

REFERÊNCIAS

LINK AIRFLOW -

https://airflow.apache.org/docs/apache-airflow/stable/?utm_source=chatgpt.com

LINK VIDEO AULA -

https://airflow.apache.org/docs/apache-airflow/stable/?utm_source=chatgpt.com